Multi-User Chat System

Project brief - Python socket clients, simple live server monitoring through WebSockets, and an optional relay.

1. Project Objective

Develop a multi-user chat system that supports public and private messaging between Python TCP clients. Extend the Echo or Simple Chat module from Project 01 to practice socket programming, basic concurrency, connection management, logging, and rate limiting.

Implement one Python chat-client application and run multiple instances of it. Use WebSockets only to update a simple server-monitoring page in the browser; a browser chat client is not required.

Component	Requirement
Chat server	Required. Accepts multiple Python TCP socket clients and manages connected users and message routing.
Python chat client	Required. A separate Python application using TCP sockets, with either a console interface or a GUI.
Server monitoring page	Required. A simple read-only HTML/JavaScript page receiving live server events and logs through a WebSocket connection on a separate monitoring port.
Chat relay	Optional. A separate intermediary for Python TCP clients that adds the reserved * nickname prefix.

Required tools and prerequisites

- Python 3 for the server, Python client, and optional relay. A simple socket and threading implementation, such as one thread per client, is sufficient and eligible for full marks. Other concurrency approaches are allowed but not required.
- Basic HTML/JavaScript for the monitoring page. A GUI framework such as tkinter is optional for the Python client.
- HTTP and WebSocket libraries are permitted; list all dependencies and versions in the README. Implement the core Python TCP chat connection using sockets.
- Windows, macOS, or Linux. A virtual environment may be used to isolate dependencies.

Interface expectations

A console Python client and a plain monitoring page can earn full marks. The monitor may be a scrolling text area, a list, or a basic table. A dashboard, charts, elaborate styling, and browser-based chat controls are not required.

AI use and academic integrity

AI assistance is permitted ONLY for the visual user interface (UI). Any AI use outside this scope means the project CANNOT RECEIVE FULL MARKS, even with bonus points. Plagiarism results in ZERO (0/100) for the entire project. See Section 3.3.

2. Chat Server

2.1 Listening interfaces and shared state

- Allow configuration of listening IP address(es), the TCP chat port, and the monitoring HTTP/WebSocket port(s), with documented defaults.
- Serve monitoring HTTP and WebSocket on ports separate from all chat-service ports. They may share one monitoring port or use two different ports. Using only a different URL path on a chat port is not sufficient.
- Maintain a shared nickname registry for connected Python chat clients. Monitoring viewers are not chat users and must not appear in the chat user list.

2.2 Nickname assignment and the reserved prefix

- Ensure nicknames are unique across all connected chat users. If a requested name is taken, append a suffix and recheck uniqueness. Return the assigned nickname to the client.
- Clients must not choose a nickname beginning with *. Only the relay may add this prefix to identify a relayed user. Direct TCP clients must be prevented from registering themselves with that prefix.
- If the optional relay is implemented, distinguish its traffic from direct chat connections. A separate server-side relay port is sufficient; relay authentication, authorization, and tokens are not required.
- Accept *nickname through the recognized relay path. If a collision requires a suffix, preserve the leading *, for example *User123. Make nickname checking and registration atomic under concurrent requests.

2.3 Public and private messaging

- For each public message, the server must send a separate copy to every connected chat user, including the sender, over that user's existing TCP connection. This is application-layer message distribution; do not use IP broadcast or multicast. Include the assigned source nickname and a server-generated timestamp in each delivered message.
- Deliver private message content only to the intended recipient among other chat users. A sender-side echo or delivery confirmation is allowed. Never forward private content to unrelated chat clients.
- If the recipient is not currently connected, return a clear error to the sender. Do not queue or store the message for later delivery. Recording the failed attempt in the server log is allowed.
- Notify all chat clients when users join or leave and keep their user lists synchronized.

2.4 Logging, monitoring, and rate limiting

- Write public/private message logs to a local text file, including timestamp, sender, message type, and recipient where applicable. Also record connections, disconnections, errors, and rate-limit events. Console output alone is not sufficient; no database is required.
- Use WebSockets to append live events to a simple read-only monitoring page, including public/private message details and connection/error events. Show events received after the page connects; loading previous logs is not required. A text area, list, or basic table is sufficient.
- Keep monitoring HTTP/WebSocket separate from chat ports. Ordinary chat connections receive only chat data. A monitoring login, token, or user-role system is not required.
- Display the current chat-user count and processed-message count as text. Increment the message count by 1 for each public/private message received and forwarded by the server, regardless of recipient count. Exclude rejected messages and system events. Update the counts when relevant events occur.
- Apply a documented per-user rate limit to public and private messages. On excess traffic, temporarily mute or disconnect the user and report the reason. See Section 5 for basic functional tests.

3. Python Chat Client and Optional Relay

3.1 Python TCP client - required

- Prompt for a nickname and server address/port, or accept these as command-line arguments. Connect directly to the chat server or, when implemented, through the optional relay.
- Display public messages with timestamps and source nicknames, maintain the current user list, and support sending and receiving private messages.
- For a GUI, selecting or double-clicking a user may open a private-chat window. Incoming private messages may open or activate that conversation.
- For a console interface, allow recipient selection through a numbered user list, a command argument, or an explicit command such as /msg nickname message. Display incoming private messages with clearly identified sender and private-message status. Separate windows and double-clicking are not required.
- Support an Exit command/button and show a clear message when the server or relay becomes unavailable. Receiving messages must remain possible while the user is entering input.

3.2 Chat relay - optional

The relay path, if attempted, is Python TCP client to relay to main server. The monitoring connection is separate and does not pass through the relay.

- Listen on a configurable local address/port and create a corresponding upstream connection to the main server for each accepted client.
- Reject a client-requested nickname beginning with *. For a valid initial registration, add exactly one leading *, then forward the registration through the recognized relay path.
- Parse enough of the registration exchange to rewrite the nickname correctly even when data arrives in fragments. After registration and any relay setup, forward chat bytes unchanged in both directions.
- Forward the server's assigned nickname back to the client. Support multiple relayed users concurrently; one user must not block another.
- If either side closes or fails, close the paired connection and release associated resources. Optional local relay logging is allowed.

3.3 AI use and academic integrity - entire project

AI tools may be used ONLY for visual UI design and presentation: HTML/CSS, layout, styling, GUI widgets, and displaying data already provided to the UI. This applies to the Python client interface and monitoring page, but not their networking or application logic.

Non-UI work must be completed without AI assistance, including design, code generation/completion, explanation, review, debugging, and modification. This covers sockets/WebSockets, message handling, threading, users, routing, relay logic, rate limiting, file logging, counters, tests, and documentation. Network/application logic remains non-UI work even inside a UI file or event handler.

Copying any part of another student's or a third-party project solution is prohibited, even if cited or superficially modified. Renaming, reformatting, or translating copied work does not make it original. Instructor-provided examples and explicitly permitted libraries may be used; identify reused course examples in the README.

NON-UI AI USE WILL REDUCE THE GRADE: THE PROJECT CANNOT RECEIVE FULL MARKS (100/100), EVEN WITH BONUS POINTS. The instructor sets the deduction according to the extent of non-permitted assistance. Permitted UI-only AI use does not itself reduce the grade; identify the tool and affected UI sections in the README. **PLAGIARISM IN ANY PART RESULTS IN ZERO (0/100) FOR THE ENTIRE PROJECT, REGARDLESS OF BONUSES.**

4. Integration and Robustness

4.1 Application structure and execution

Use separate entry points such as chat_server.py, chat_client.py, and optional chat_relay.py, with an HTML/JavaScript page for monitoring. An integrated launcher/menu is also acceptable if the components can be run independently.

Connection path	Purpose
Python client to TCP chat server	Direct socket-based chat.
Monitoring page to monitoring WebSocket port	Live events and counts, separate from chat ports. The page's HTTP port may be the same as or different from the monitoring WebSocket port.
Python client to relay to chat server	Optional relay path; assigned nickname starts with *.

4.2 Message handling

- Document the application message format and registration exchange. Define how TCP message boundaries are detected; do not assume a single receive call returns exactly one complete message.
- Correctly handle fragmented messages and multiple messages arriving in one read. Document text encoding and maximum accepted message size.
- Validate message type, required fields, recipient, and size. Invalid input must produce a controlled error or disconnect without crashing the server or corrupting other users' sessions.
- Derive sender identity from the registered connection. A client-supplied sender field must not allow impersonation of another user.

4.3 Concurrent operation and connection lifecycle

- A simple thread-based server is sufficient for full marks. For example, use one thread per client and basic locks where shared state or socket writes need protection. Asyncio, select, thread pools, and advanced queue designs are not required.
- Protect shared user state, nickname allocation, counters, and log writes against concurrent updates. Preserve complete outbound message boundaries when multiple tasks write to one connection.
- Normal connection, messaging, and Exit must work correctly. If a client closes or a socket operation reports disconnection, handle it without terminating the entire server or unrelated client connections.
- Remove disconnected users, notify remaining clients, and close their sockets when disconnection is detected. Allow the associated client-handler thread to finish.
- Custom heartbeat mechanisms, automatic reconnection, session recovery, and immediate detection of silent network failures are not required.
- Keep multiple clients and the monitoring page responsive during normal use. Closing the monitoring page must not interrupt chat service.
- Advanced slow-client handling, bounded outbound queues, and stress testing are optional improvements and are not required for full marks.

4.4 Rate-limit policy

Implement a simple per-user limit to prevent rapid repeated messages. A message counter that resets at fixed time intervals is sufficient. Choose and document the allowed count, interval, and temporary mute or disconnect behavior. Apply the limit per chat user, not globally to all users sharing a relay or IP address. Advanced rate-limiting algorithms are not required.

5. Basic Functional Tests

Demonstrate the following scenarios with short explanations and screenshots or console/server logs. Manual testing is sufficient; automated test scripts and a formal test report are not required. Run multiple instances of the Python client: two for ordinary chat and a third for the private-message check. All components may run on the same computer.

ID / scenario	Required result
T01 / Public chat	Connect two Python clients. Send public messages in both directions and show timestamps, assigned nicknames, and the current user list.
T02 / Private chat	Send private messages between two Python clients. Use a third Python client to show that unrelated users do not receive the private content. Attempt to message a disconnected nickname and show the error.
T03 / Nicknames	Connect two clients requesting the same nickname and show that different names are assigned. Show that direct clients cannot choose a nickname beginning with *; simultaneous connection attempts are not required.
T04 / Disconnection	Use Exit, and also close a client window or stop that client normally. The server must remain available. Show user-list cleanup after the server detects disconnection; simulated network failures are not required.
T05 / Rate limiting	Send messages rapidly from one Python client and show the configured mute or disconnect behavior.
T06 / Logs and live monitoring	Show the local log file and the monitoring page. Show new public/private message events and counts updating without manual refresh. A message sent to multiple users must increase the message count by only 1. Close the page and verify that chat still works.
T07 / Relay (optional)	Connect a Python client through the relay. Show the assigned * prefix and message exchange with a directly connected client. Exit the relayed client and show that its connection is cleaned up.

Optional additional checks

You may also test fragmented or combined TCP messages, malformed input, abrupt disconnections, slow receivers, and heavier traffic. These additional tests are encouraged but are not mandatory and do not carry separate required test points. The application must still implement the message-handling rules in Section 4.2.

6. Deliverables and Evaluation

6.1 Deliverables

- **Source code:** server, Python client, simple monitoring page, optional relay, and supporting modules. Include a dependency file where applicable.
- **README and execution guide:** installation, startup order, configurable addresses/ports, browser URLs, private-message controls, monitoring access, and optional relay setup. Include example commands.
- **Design notes:** briefly describe the application message format and framing, concurrency model, nickname handling, relay connection setup if implemented, monitoring port(s), log-file location, and rate-limit policy. These notes may be included in the README.
- **Test evidence:** short explanations and selected screenshots or logs for T01-T06, and T07 if the relay is submitted. Manual demonstrations are sufficient; automated scripts, stress tests, and a separate detailed test report are not required.

6.2 Assessment rubric - 100 points

Criterion	Points	What is assessed
TCP communication and protocol	20	Normal connection and registration (5), message framing (7), basic message validation (4), and using the registered sender identity (4).
Core chat functionality	20	Unique nicknames (5), public routing (5), private routing (5), and synchronized user lists (5).
Basic concurrency and disconnection	25	Multiple clients exchanging messages correctly with safe shared state/writes (10), normal Exit and user-list cleanup (8), and the server continuing when one client closes (7). Simple threading can receive all 25 points.
Logging and live monitoring	10	Message/event logs written to a local text file (4), live WebSocket updates on a monitoring port separate from chat (4), and correct counts (2).
Rate limiting	10	Explicit policy (3), correct server-side enforcement (5), and observable notification/logging (2).
Basic functional tests	10	Coverage of the six required functional scenarios (7), with understandable explanations and screenshots/logs (3).
Code quality and documentation	5	Readable modular code, clear execution guide, dependencies, and explanations of design choices.

Optional relay bonus - up to 10 points

Bonus: prefix handling and connection setup (3), concurrent bidirectional forwarding (4), and cleanup with test evidence (3). Authentication is not required. Omitting the relay does not reduce the base score. Score before academic-integrity adjustments = min(100, base score + relay bonus).

A correct thread-based server, console client, and plain monitoring page can earn full marks, subject to Section 3.3. Non-UI AI use prevents full marks even with bonuses; plagiarism gives the entire project 0/100.

6.3 Suggested development order

Build TCP chat first, then private messaging and user lists. Add WebSocket monitoring and rate limiting. Complete basic tests and documentation before the optional relay. Use version control throughout.